

Lakeside Hotel Booking Website - Comprehensive Documentation

Table of Contents

- 1. Introduction
- 2. Architecture 2.1. Project Structure 2.2. Key Components 2.3. Security Architecture
- 3. Entities (Models)
- 4. Repositories
- 5. Services
- 6. Controllers
- 7. Security Configuration
- 8. API Endpoints Details
- 9. Exception Handling
- 10. Database Schema
- 11. Setup and Deployment 11.1. Prerequisites 11.2. Database Configuration 11.3. Building and Running
- 12. Future Enhancements

1. Introduction

The Lakeside Hotel Booking Website is a robust backend application designed to manage hotel operations, including room inventory, guest bookings, user authentication, and role-based access control. Built with Spring Boot, it provides a secure and scalable foundation for a modern hotel management system.

2. Architecture

The application follows a layered architecture, common in Spring Boot applications, separating concerns into distinct components: Controller, Service, and Repository layers.

2.1. Project Structure

├─ .mvn/ └─ src/ | └─ main/ | | └─ java/com/dailycodework/lakesidehotel/ | | | └─ LakeSideHotelApplication.java // Main Spring Boot application | | | └─ controller/ // REST API endpoints | | | └─ exception/ // Custom exceptions | | | └─ model/ // JPA entities (database models) | | | └─ repository/ // Data access layer | | | └─ request/ // DTOs for incoming requests | | | └─ response/ // DTOs for outgoing responses | | | └─ security/ // Spring Security configurations, JWT utilities | | | └─ jwt/ | | | | └─ user/ | | | | └─ service/ // Business logic layer | | | └─ resources/ | | | └─ application.properties // Application configuration (e.g., database, JWT) | | | └─ application.yml // Alternative application configuration | | | └─ test/ | | | └─ java/com/dailycodework/lakesidehotel/ | | | └─ LakeSideHotelApplicationTests.java // Unit/Integration tests | | | └─ pom.xml // Maven Project Object Model | | | └─ README.md

2.2. Key Components

- **Models (`com.dailycodework.lakesidehotel.model`)**: JPA entities representing the database tables. These include `User`, `Role`, `Room`, and `BookedRoom`. They define the data structure and relationships.
- **Repositories (`com.dailycodework.lakesidehotel.repository`)**: Interfaces extending `JpaRepository` that provide basic CRUD operations and custom query methods for interacting with the database.
- **Services (`com.dailycodework.lakesidehotel.service`)**: Contain the core business logic. They interact with repositories to perform operations and apply business rules before returning data to controllers. Interfaces (`IBookingService`, `IRoomService`, `IUserService`, `IRoleService`) are used for abstraction and dependency inversion.
- **Controllers (`com.dailycodework.lakesidehotel.controller`)**: RESTful endpoints that handle incoming HTTP requests, delegate tasks to services, and return appropriate HTTP responses.
- **Security (`com.dailycodework.lakesidehotel.security`)**: Manages user authentication, authorization, and JWT token handling.

2.3. Security Architecture

The application implements a robust security layer using Spring Security and JWT.

- **`WebSecurityConfig.java`**: Configures the security filter chain. It sets up `Csrf` disabling, exception handling for unauthorized access, session management (stateless for JWT), and request authorization rules. It also integrates custom authentication provider and JWT token filter.
- **`HotelUserDetailsService.java`**: Implements Spring Security's `UserDetailsService` to load user-specific data by email during authentication.
- **`HotelUserDetails.java`**: Implements `UserDetails` and wraps the `User` model, providing necessary user details (username, password, authorities) for Spring Security.
- **`JwtUtils.java`**: Utility class for generating, parsing, and validating JWT tokens. It handles the JWT secret and expiration.
- **`AuthTokenFilter.java`**: A custom `OncePerRequestFilter` that intercepts incoming requests, extracts and validates JWT tokens, and sets the `Authentication` object in Spring Security's `SecurityContextHolder`.
- **`JwtAuthEntryPoint.java`**: Handles authentication failures, sending an unauthorized response to the client.
- **`CorsConfig.java`**: Configures Cross-Origin Resource Sharing (CORS) to allow requests from specified origins (e.g., `http://localhost:5173` for frontend development).

3. Entities (Models)

All entities are JPA annotated for Hibernate mapping to a relational database.

- **`User.java`**: Represents a user in the system.
 - `id`: Primary key
 - `firstName`, `lastName`, `email`, `password`
 - `roles`: Many-to-Many relationship with `Role` entity. Users can have multiple roles (e.g., `USER`, `ADMIN`).
- **`Role.java`**: Defines user roles within the application.
 - `id`: Primary key
 - `name`: Role name (e.g., `"ROLE_USER"`, `"ROLE_ADMIN"`)

- `users` : Many-to-Many relationship with `User` entity, mapped by `roles` in `User`.
- **Room.java** : Represents a hotel room.
 - `id` : Primary key
 - `roomType`, `roomPrice`, `isBooked`
 - `photo` : BLOB type for storing room images.
 - `bookings` : One-to-Many relationship with `BookedRoom`.
- **BookedRoom.java** : Represents a room booking.
 - `bookingId` : Primary key
 - `checkInDate`, `checkOutDate`, `guestFullName`, `guestEmail`, `numOfAdults`, `numOfChildren`, `totalNumOfGuest`.
 - `bookingConfirmationCode` : Unique code for each booking.
 - `room` : Many-to-One relationship with `Room` entity.

4. Repositories

JPA Repositories for data access.

- **BookingRepository.java** : Extends `JpaRepository` for `BookedRoom` entity.
 - `findByRoomId(Long roomId)`
 - `findByBookingConfirmationCode(String confirmationCode)`
 - `findByGuestEmail(String email)`
- **RoleRepository.java** : Extends `JpaRepository` for `Role` entity.
 - `findByName(String role)`
 - `existsByName(String role)`
- **RoomRepository.java** : Extends `JpaRepository` for `Room` entity.
 - `findDistinctRoomTypes()` : Custom query to get unique room types.
 - `findAvailableRoomsByDatesAndType(...)` : Custom query for finding available rooms based on dates and type.
- **UserRepository.java** : Extends `JpaRepository` for `User` entity.
 - `existsByEmail(String email)`
 - `deleteByEmail(String email)`
 - `findByEmail(String email)`

5. Services

Implement business logic and interact with repositories.

- **BookingService.java** : Manages room bookings.
 - `saveBooking()` : Validates booking dates and room availability before saving.
 - `roomIsAvailable()` : Helper method to check if a room is available for the requested dates.
- **RoomService.java** : Handles room-related operations.
 - `addNewRoom()` : Saves new room details, including photo.
 - `getRoomPhotoByRoomId()` : Retrieves room photo as byte array.
- **UserService.java** : Manages user accounts.
 - `registerUser()` : Encodes password and assigns "ROLE_USER" to new users.
- **RoleService.java** : Manages roles and user-role assignments.
 - `createRole()` : Creates a new role, ensuring it doesn't already exist.

- `assignRoleToUser()` and `removeUserFromRole()` : Handle associating/disassociating users with roles.

6. Controllers

RESTful APIs exposing the application's functionality.

- **AuthController.java** : Handles user registration and login.
 - `/auth/register-user` : Endpoint for new user registration.
 - `/auth/login` : Endpoint for user authentication, returning a JWT token upon successful login.
- **BookingController.java** : Manages booking-related API calls.
 - `/bookings/room/{roomId}/booking` : Book a room.
 - `/bookings/confirmation/{confirmationCode}` : Retrieve booking by confirmation code.
 - `/bookings/all-bookings` : Get all bookings (ADMIN access).
 - `/bookings/user/{email}/bookings` : Get bookings by user email.
- **RoomController.java** : Handles room management.
 - `/rooms/add/new-room` : Add new room (ADMIN access).
 - `/rooms/available-rooms` : Find rooms available for specific dates and type.
- **UserController.java** : Manages user-related API calls.
 - `/users/all` : Get all users (ADMIN access).
 - `/users/{email}` : Get user by email.
- **RoleController.java** : Manages role-related API calls.
 - `/roles/create-new-role` : Create a new role.
 - `/roles/assign-user-to-role` : Assign a role to a user.

7. Security Configuration

The `WebSecurityConfig` class is central to defining security rules:

- **Stateless Session**: `SessionCreationPolicy.STATELESS` ensures that sessions are not used to store user's state, making the application suitable for REST APIs with JWT.
- **Request Authorization**:
 - `/auth/**`, `/rooms/**`, `/bookings/**` are permitted for all.
 - `/roles/**` requires `ROLE_ADMIN`.
 - Any other request requires authentication (`.anyRequest().authenticated()`).
- **Method Level Security**: `@EnableMethodSecurity(securedEnabled = true, jsr250Enabled = true, prePostEnabled = true)` enables annotation-based security, allowing fine-grained control with `@PreAuthorize`, `@PostAuthorize`, `@Secured`, and `@RolesAllowed`. Examples are seen in `RoomController` for `addNewRoom` (`@PreAuthorize("hasRole('ROLE_ADMIN')")`) and `UserController` for `getUsers`.

8. API Endpoints Details

All endpoints are prefixed with the server port `9192`.

Authentication Endpoints (`/auth`)

- **POST /auth/register-user**
 - **Description:** Registers a new user with a default `ROLE_USER`.
 - **Request Body:** `LoginRequest` (email, password)
 - **Responses:** `200 OK` (Registration successful), `409 Conflict` (User already exists).
- **POST /auth/login**
 - **Description:** Authenticates a user and returns a JWT token.
 - **Request Body:** `LoginRequest` (email, password).
 - **Responses:** `200 OK` (`JwtResponse` with token and user roles), `401 Unauthorized` (Invalid credentials).

Rooms Endpoints (/rooms)

- **POST /rooms/add/new-room**
 - **Description:** Adds a new room with a photo. Requires `ROLE_ADMIN`.
 - **Request Params:** `photo` (`MultipartFile`), `roomType` (`String`), `roomPrice` (`BigDecimal`).
 - **Responses:** `200 OK` (`RoomResponse`), `500 Internal Server Error` (e.g., photo processing issues).
- **GET /rooms/all-rooms**
 - **Description:** Retrieves all rooms available in the hotel.
 - **Responses:** `200 OK` (List of `RoomResponse` objects).
- **GET /rooms/available-rooms**
 - **Description:** Finds rooms that are available between specified check-in and check-out dates for a given room type.
 - **Request Params:** `checkInDate` (`LocalDate`), `checkOutDate` (`LocalDate`), `roomType` (`String`).
 - **Responses:** `200 OK` (List of `RoomResponse` objects), `204 No Content` (No available rooms).

Bookings Endpoints (/bookings)

- **POST /bookings/room/{roomId}/booking**
 - **Description:** Creates a new booking for a specified room.
 - **Request Body:** `BookedRoom` (`checkInDate`, `checkOutDate`, `guestFullName`, `guestEmail`, `numOfAdults`, `numOfChildren`).
 - **Responses:** `200 OK` (Booking confirmation code), `400 Bad Request` (Invalid dates or room not available).
- **GET /bookings/confirmation/{confirmationCode}**
 - **Description:** Retrieves booking details using a unique confirmation code.
 - **Responses:** `200 OK` (`BookingResponse`), `404 Not Found` (Booking not found).
- **GET /bookings/all-bookings**
 - **Description:** Retrieves all bookings in the system. Requires `ROLE_ADMIN`.
 - **Responses:** `200 OK` (List of `BookingResponse` objects).
- **GET /bookings/user/{email}/bookings**
 - **Description:** Retrieves all bookings made by a specific user email.
 - **Responses:** `200 OK` (List of `BookingResponse` objects).
- **DELETE /bookings/booking/{bookingId}/delete**
 - **Description:** Cancels a booking by its ID.
 - **Responses:** `200 OK` (No Content).

Users Endpoints (/users)

- **GET /users/all**
 - **Description:** Retrieves all registered users. Requires `ROLE_ADMIN`.
 - **Responses:** `200 OK` (List of `User` objects).
- **GET /users/{email}**
 - **Description:** Retrieves a user by their email. Requires `ROLE_USER` or `ROLE_ADMIN`.
 - **Responses:** `200 OK` (`User` object), `404 Not Found` (User not found).
- **DELETE /users/delete/{userId}**
 - **Description:** Deletes a user by their email (`userId` parameter actually takes email). Requires `ROLE_ADMIN` or the user deleting themselves.
 - **Responses:** `200 OK` (Success message), `404 Not Found` (User not found), `500 Internal Server Error` (Other errors).

Roles Endpoints (/roles)

- **GET /roles/all-roles**
 - **Description:** Retrieves all available roles.
 - **Responses:** `200 OK` (List of `Role` objects).
- **POST /roles/create-new-role**
 - **Description:** Creates a new role. Requires `ROLE_ADMIN`.
 - **Request Body:** `{"name": "..."}.`
 - **Responses:** `200 OK` (Success message), `409 Conflict` (Role already exists).
- **DELETE /roles/delete/{roleId}**
 - **Description:** Deletes a role. Requires `ROLE_ADMIN`.
 - **Responses:** `200 OK` (No Content).
- **POST /roles/remove-all-users-from-role/{roleId}**
 - **Description:** Removes all users from a specific role. Requires `ROLE_ADMIN`.
 - **Responses:** `200 OK` (`Role` object with no users).
- **POST /roles/remove-user-from-role**
 - **Description:** Removes a specific user from a role. Requires `ROLE_ADMIN`.
 - **Request Params:** `userId` (Long), `roleId` (Long).
 - **Responses:** `200 OK` (`User` object).
- **POST /roles/assign-user-to-role**
 - **Description:** Assigns a role to a user. Requires `ROLE_ADMIN`.
 - **Request Params:** `userId` (Long), `roleId` (Long).
 - **Responses:** `200 OK` (`User` object), `409 Conflict` (User already has the role).

9. Exception Handling

The application defines custom exception classes to provide meaningful error messages:

- **ResourceNotFoundException.java** : Thrown when a requested resource (e.g., room, booking) is not found.
- **InvalidBookingRequestException.java** : Thrown for invalid booking details (e.g., check-in date after check-out date, room not available).

- **UserAlreadyExistsException.java** : Thrown during user registration if the email already exists.
- **RoleAlreadyExistException.java** : Thrown when trying to create a role that already exists.
- **InternalServerErrorException.java** : A generic exception for unhandled server-side issues.
- **PhotoRetrievalException.java** : Thrown when there's an issue retrieving room photos.

These exceptions are caught by `@ExceptionHandler` methods (implicitly or explicitly handled in controllers) to return appropriate HTTP status codes and error messages to the client.

10. Database Schema

The database schema is derived from the JPA entities (`User` , `Role` , `Room` , `BookedRoom`) and their relationships. The `ddl-auto=update` property in `application.properties` or `application.yml` allows Hibernate to automatically create or update the schema based on entity definitions.

Tables:

- **ROOM :**
 - `id` (BIGINT, Primary Key)
 - `room_type` (VARCHAR)
 - `room_price` (DECIMAL)
 - `is_booked` (BOOLEAN)
 - `photo` (BLOB)
- **BOOKED_ROOM :**
 - `booking_id` (BIGINT, Primary Key)
 - `check_in` (DATE)
 - `check_out` (DATE)
 - `guest_full_name` (VARCHAR)
 - `guest_email` (VARCHAR)
 - `adults` (INT)
 - `children` (INT)
 - `total_guest` (INT)
 - `confirmation_code` (VARCHAR, Unique)
 - `room_id` (BIGINT, Foreign Key referencing `ROOM.id`)
- **USERS :** (Note: Table name is `users` based on `@Table(name = "users")` in `User.java`)
 - `id` (BIGINT, Primary Key)
 - `first_name` (VARCHAR)
 - `last_name` (VARCHAR)
 - `email` (VARCHAR, Unique)
 - `password` (VARCHAR)
- **ROLE :**
 - `id` (BIGINT, Primary Key)
 - `name` (VARCHAR, Unique)
- **USER_ROLES :** (Join table for Many-to-Many relationship between `USERS` and `ROLE`)
 - `user_id` (BIGINT, Primary Key, Foreign Key referencing `USERS.id`)

- `role_id` (BIGINT, Primary Key, Foreign Key referencing `ROLE.id`)

11. Setup and Deployment

11.1. Prerequisites

- **Java Development Kit (JDK) 21 or higher:** Ensure you have a compatible JDK installed.
- **Apache Maven:** Version 3.6 or higher for building the project.
- **Database Server:** MySQL (default) or PostgreSQL installed and running.

11.2. Database Configuration

- **MySQL:**
 - Create a new database. For example, using MySQL client:

```
CREATE DATABASE lakeSide_hotel_db;
```
 - Open `src/main/resources/application.properties`.
 - Update `spring.datasource.url`, `spring.datasource.username`, and `spring.datasource.password` to match your MySQL setup.
 - The `ddl-auto=update` property will automatically create/update tables on application startup.
- **PostgreSQL:**
 - Comment out the `spring.datasource` properties in `application.properties`.
 - Uncomment and configure the `spring.datasource` properties in `src/main/resources/application.yml`.
 - Ensure the PostgreSQL driver dependency is added to `pom.xml` instead of `mysql-connector-j`.

11.3. Building and Running

1. Clone the Repository:

```
git clone [https://github.com/suryanshkalchuri/hotel-booking-website.git]
(https://github.com/suryanshkalchuri/hotel-booking-website.git)
cd hotel-booking-website/Hotel-Booking-Website-
83623a2d97535b6bd012a239d470845ad5cf8819
```

(Note: Adjust path if your local clone structure differs.)

2. Build with Maven:

```
mvn clean install
```

This command compiles the source code, runs tests, and packages the application into a JAR file.

3. Run the Application:

```
mvn spring-boot:run
```

Alternatively, you can run the generated JAR file:

```
java -jar target/lakeSide-hotel-0.0.1-SNAPSHOT.jar
```

The application will start on `http://localhost:9192`.

12. Future Enhancements

- **Payment Gateway Integration:** Integrate with a third-party payment provider (e.g., Stripe, PayPal).
- **Email Notifications:** Send automated email confirmations for bookings, cancellations, etc.
- **Admin Dashboard:** Develop a web-based admin interface for easier management of rooms, bookings, and users.
- **User Profiles:** Allow users to view and manage their personal details and booking history.
- **Room Reviews/Ratings:** Implement a system for guests to leave reviews and ratings for rooms.
- **Advanced Search Filters:** Add more sophisticated search options for rooms (e.g., amenities, guest capacity, price range).
- **Logging and Monitoring:** Implement comprehensive logging and integrate with monitoring tools (e.g., Prometheus, Grafana).
- **Containerization:** Dockerize the application for easier deployment and scalability.
- **Cloud Deployment:** Deploy the application to cloud platforms like AWS, Google Cloud, or Azure.